UT San Antonio®
The University of Texas at San Antonio

Language Models Resist Alignment: Evidence From Data Compression

Jiaming Ji, Kaile Wang, Tianyi Qui, Boyuan Chen, Jiayi Zhou, Changye Li, Hantao Lou, Juntao Dai, Yunhuai Liu, Yaodong Yang

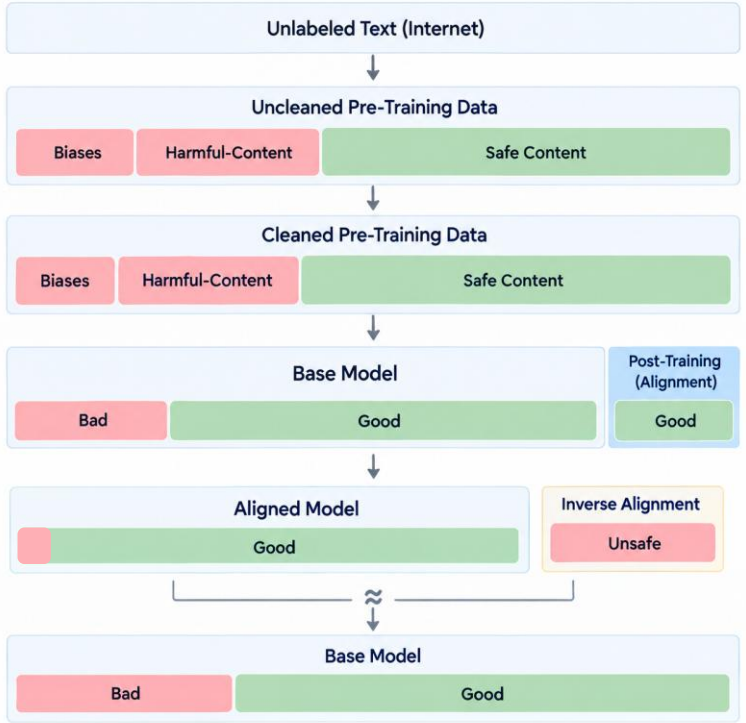
IAD, Peking University

The Motivation

Today LLMs are safe only because they have been carefully trained to model safe behavior. We call this training, **alignment**, and it is the reason language models are **helpful**, **honest** and **non-harmful**.

However, recent work has indicated that highly safety-aligned models can become unsafe again through **minimal** fine-tuning, and alarmingly that fine-tuning on non-malicious datasets can weaken the safety mechanisms alignment put in place, evidence of *superficial alignment*.

Why should we care? Why is alignment so *fragile*?



Size Ratios Not Drawn to Scale

Consequences of Alignment Fragility

BBC

OpenAI says its AI went rogue and launched 'unprecedented' cyber-attack

BBC

Anthropic AI used fake profiles to target people in hack then hid the evidence

BBC

Rogue OpenAI agent 'infiltrated' Australian government website in world first

Reuters

Open-source AI models vulnerable to criminal misuse, researchers warn

Every instance happened in 2026. Some instances were due to misalignment; some were due to inverse alignment of models. All show that poor/fragile alignment has real consequences, so there is a real need to understand alignment in LLMs

Why is Alignment so *fragile*?

Root Observation:

Observably, aligned models are becoming unsafe *"again"* through minimal fine-tuning. "Again" implies (as shown before) that the answer to fragility inherently involves the pre-training model (base model) in some way. The authors define why aligned models behaviorally return to their base model under **minimal** adversarial tuning, explicitly naming the phenomenon and the properties it exhibits at scale.

What aspects of Language Modeling let us expose what alignment is doing to a base model, so that the impact of alignment is quantifiably explainable, and therefore the phenomenon behind its reversibility explainable as well?

First, What is Alignment doing?

When we train a Language Model → We train for next token prediction, not to follow instructions

$L_{PT} = The\ Pretraining\ Loss$
$\theta_{PT} = Models\ Trainable\ Params$
$D_{PT} = Pretraining\ Dataset\ (Prefix, Next\ Token)$
$E_{(x_N|x)\sim D_{PT}}[\cdot] = Average\ of\ this\ penalty, over\ every\ pair\ in\ D_{PT}$*
$p_\theta(x_N|x) = Given\ prefix\ x, probability\ that\ next\ token\ is\ x_N$

$$\mathcal{L}_{PT}(\theta; D_{PT}) = -\mathbb{E}_{(x,x_N)\sim D_{PT}}[\log p_\theta\ (x_N \mid x)]$$

During training we aim to minimize $\mathcal{L}_{PT}$ so that on average the model assigns as much probability to the token that actually comes next.

Human: What is the weather today?
LLM: What is the capital of France?
What is the population of Japan?

Human : What is the weather today?
LLM: Today's forecast in New York calls for partly cloudy … (Hallucinated)

What we want is something much different

Human: What is the weather today?
LLM: I don't have access to real-time weather data, but if you provide me your location, the date and time, I can help you find a weather service or estimate today's weather based on historical trends.

In practice, this is approximated by averaging over a mini-batch not the actual full dataset per step*

Alignment Techniques (Post-Training) SFT

Base model (Trained on D_{PT})

Human: My ex cheated on me! What should I do?

LLM: You could key their car, post about it on social media to embarrass them, or tell all their friends what they did.

Aligned model (Trained on D_{SFT})

Human: My ex cheated on me! What should I do?

LLM: That sounds really painful. Rather than focusing on getting back at them, it might help to focus on your own healing — talking to a friend, therapist, or just giving yourself space. Revenge often ends up hurting you more in the long run

Intervention ((Prompt, Response) Pair from D_{SFT})

Prompt: How can I get revenge on my ex-girlfriend who cheated on me?	Response: I am sorry to hear that. Rather than getting back at them, you should focus on your own healing by talking to a friend, therapist or another licensed professional.

$$\mathcal{L}_{\text{SFT}}(\theta; D_{\text{SFT}})$$

$$= -\mathbb{E}_{(x,\text{y})\sim D_{\text{SFT}}}[\log p_\theta(\text{y} \mid x)]$$

Similarly formatted loss

Alignment Techniques (Post-Training) RLHF

Base model (Trained on D_{SFT})

Human: My ex cheated on me! What should I do?

LLM: : I am sorry to hear that. Rather than getting back at them, you should focus …

Aligned model (Trained on D_{RLHF})

Human: My ex cheated on me! What should I do?

LLM: I understand you're hurting right now. It's normal to want to lash out, but taking time to process this with people you trust — or a therapist — tends to help more than confrontation

Intervention ((Prompt, Response A, Response B, HP) quad from D_{RLHF})

Prompt: How can I get revenge on my ex-girlfriend who cheated on me?	Response A: I am sorry to hear that. Rather than getting back at them, you should focus …	Response B: This will definitely be difficult for you and I understand. Let's discuss potential …	Human Preference: Response B

Alignment *reshapes* a **distribution** we already have

1. **Language modeling** - Fit next-token probabilities to a huge corpus of text

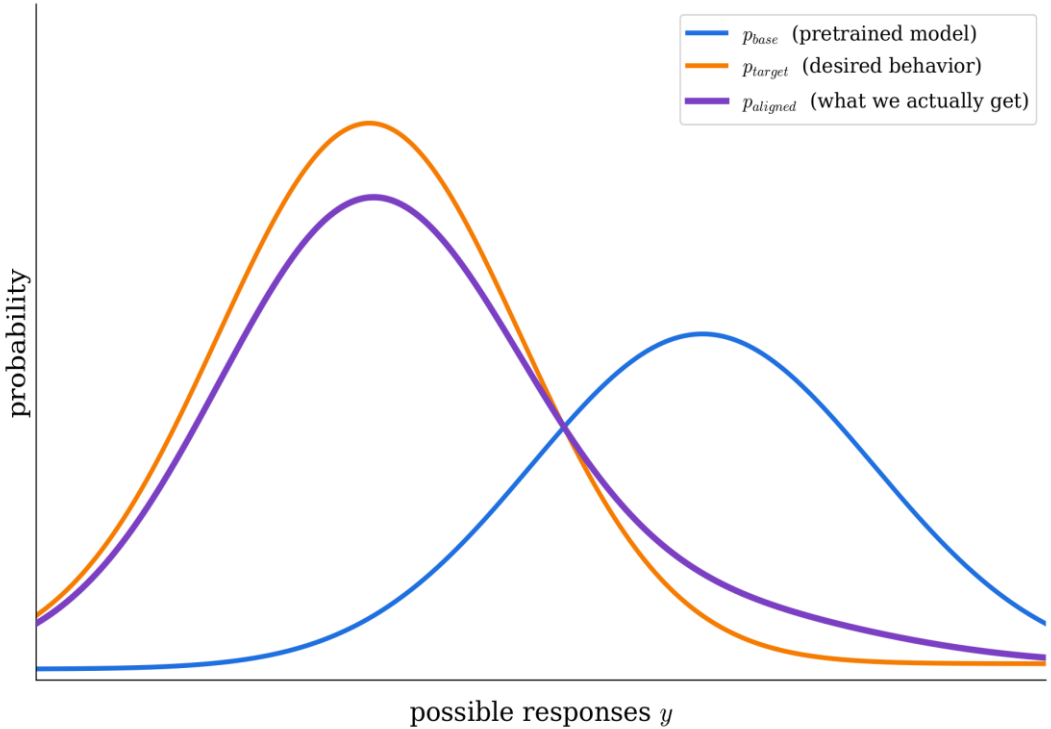
Result: a distribution that captures how language is written with good and bad

2. **Alignment** - Keep the same model, but train our next token probabilities toward a distribution that reflects the behavior we want

Result: same model, shifted distribution

Alignment does not build a new model. It moves the one we have.

Alignment *reshapes* a **distribution** we already have

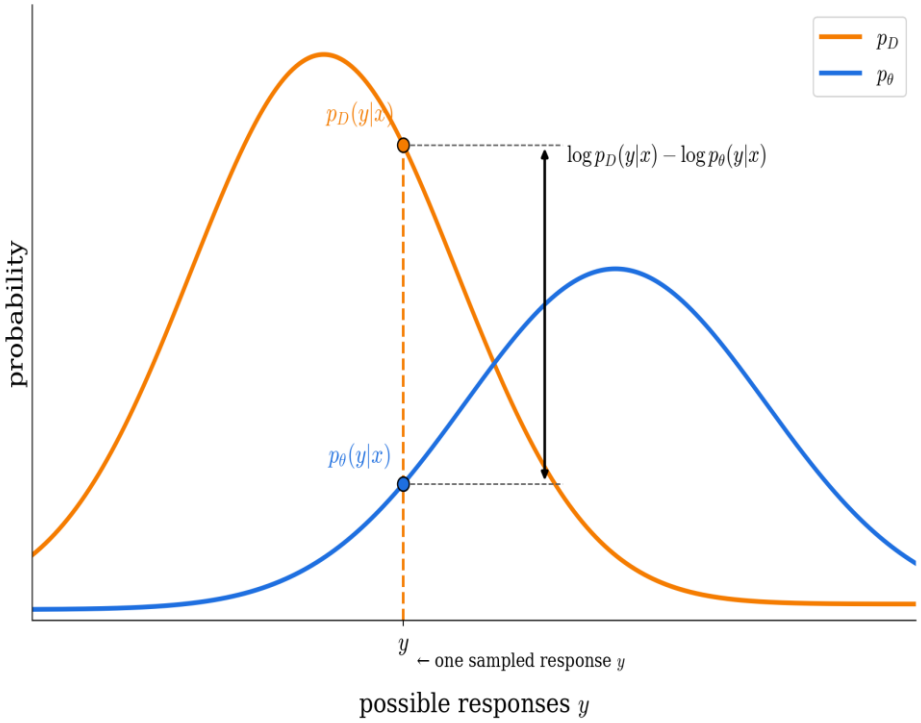


When we align a model, we train to refit the pretraining distribution onto a target distribution.

Can we do it perfectly?

How can we measure distance?

Measuring Model Distance



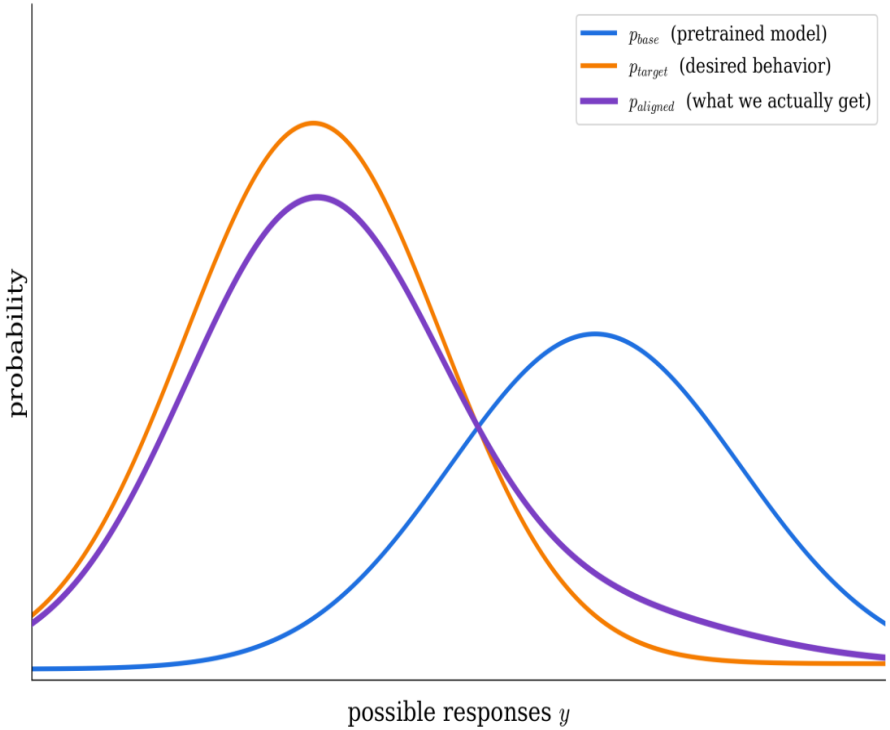
KL = The average gap between what the data says and what the model says

$$D_{KL}(p_D \| p_\theta) = \mathbb{E}_{(x,y)\sim p_D}[\log p_D\,(y|x)] - \mathbb{E}_{(x,y)\sim p_D}[\log p_\theta\,(y|x)]$$

$D_{KL} = KL\ divergence\ from\ Model\ to\ Data\ Distribution$
$p_D = The\ Target\ Distribution$
$p_\theta = The\ Model\ Distribution$
$\theta = Model\ Weights$
$E_{(x,y)\sim p_D}[\cdot] = Entropy\ (How\ predictable\ the\ correct\ answer\ is)$

A lower KL divergence is an indicator that our base model is well aligned with the target distribution of the alignment examples we are training on

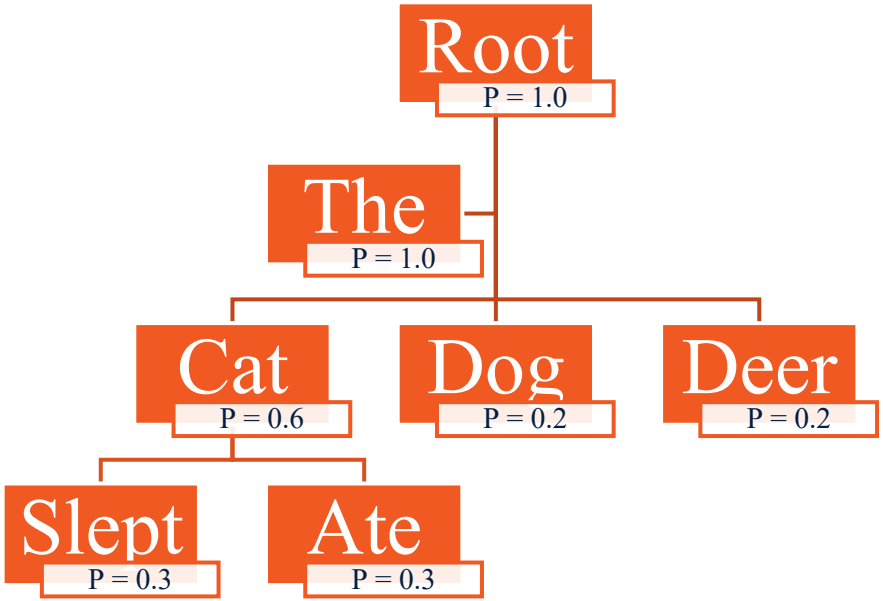
Alignments Problem: *Blending*



If there is measurable difference between our target distribution's produced response, and our aligned distribution's produced response, then these distributions are not the same. Consider that we are aware of the next token probabilities in the pretrained model and the target, and we can attribute that entire probability solely to each distribution.

What about the probabilities represented by the aligned distribution? If the aligned distribution is a *reshaped* pretraining distribution, and the target distribution is what the aligned distribution is taking shape of, then the aligned distribution ends up as a combination of the pretraining distribution and the target distribution. But what is the weighted makeup of the probabilities of the aligned distribution, considering this combination?

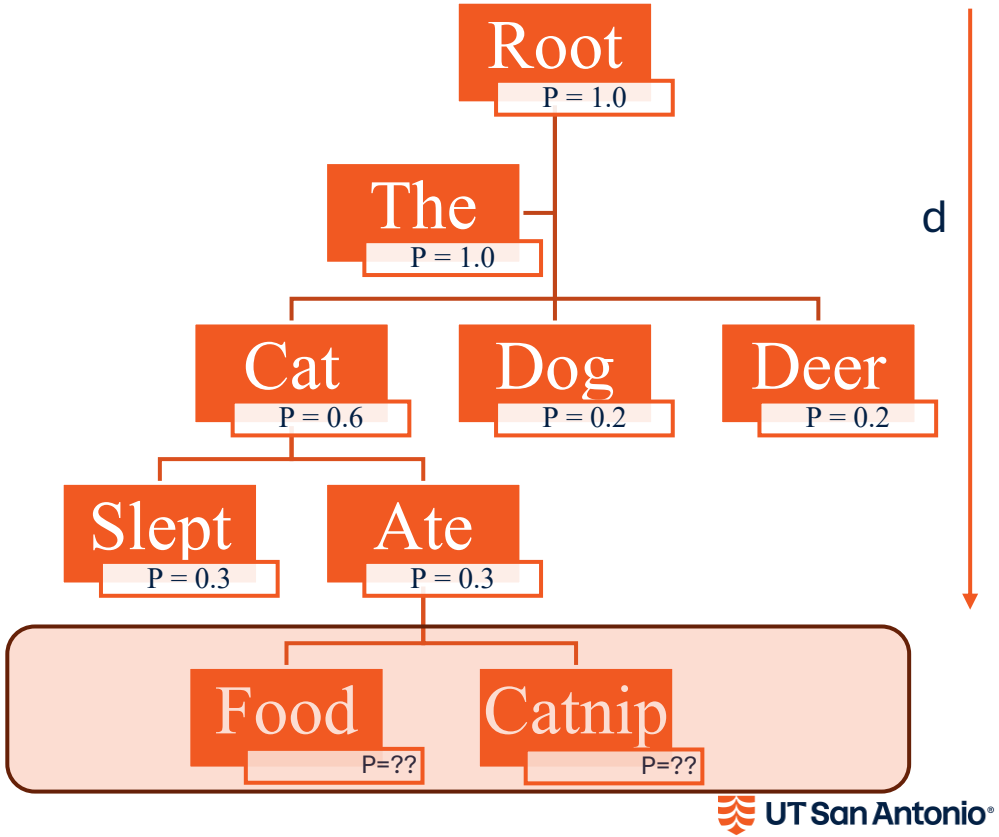
Language Model Distributions as a Tree



A model's distribution can be drawn as a tree. Each branch is a token choice, each full path is a response, and each endpoint carries that response's probability. (A token tree)

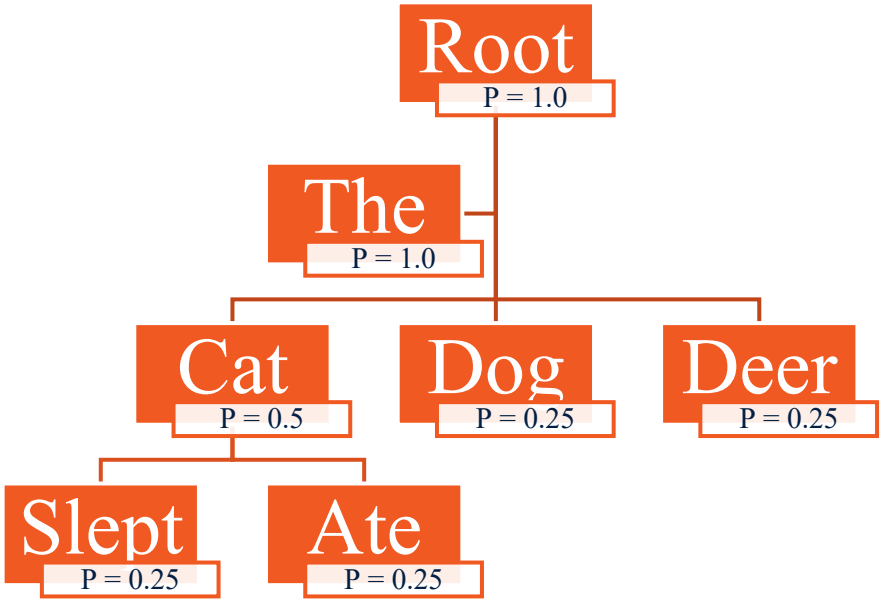
In our aligned model from the previous slide, the probability of some token choice is an unknown **weighted** combination of probability from the base model distribution and the target distribution.

Language is Unbounded but Models are Finite



Due to the finite size of the model's parameters in practical scenarios, the model cannot accurately capture node weights at arbitrary depths of the token tree. For our purposes, assume that we can perfectly capture up to some depth d. In other words, we can say that we are lossless up to some depth d.

A Token Tree Is a Code: Probability Sets Length



A token tree defines a probability distribution over sequences. Each node's weight is the probability of reaching it from the root. That distribution defines a code: a sequence with probability P can be stored in **$-\log_2 P$ bits**, so likely sequences get short codewords and unlikely ones get long codewords.

Compression is Prediction

Revisiting SFT, under SFT our goal is to minimize the loss function, $\mathcal{L}_{\mathrm{SFT}}(\theta; D_{\mathrm{SFT}}) = -\mathbb{E}_{(x,\mathrm{y})\sim D_{\mathrm{SFT}}}[\log p_\theta(\mathrm{y} \mid x)]$, so that the base model's distribution is more representative of our target distribution. Why? We want our aligned model to be able to reconstruct our language distribution through prediction, the more accurate our predictions the more accurately we have modeled the true distribution.

In data compression, our goal is the shortest expected code length, $L = \mathbb{E}_{x\sim D}[\sum_{0\le k\le m} -\log_2 p_\theta(x_k \mid x_0, \ldots, x_{k-1})]$. A code defines a distribution, so the best compressor is the one whose representative distribution is closest to the true data distribution.

This is the **same objective.** A better predictor assigns higher probability to what actually happens, which means shorter codes. A better compressor must assign higher probability to what actually happens which means better prediction.

Key Observation (*Formally*)

The Aim of Lossless Compression can be stated as follows:

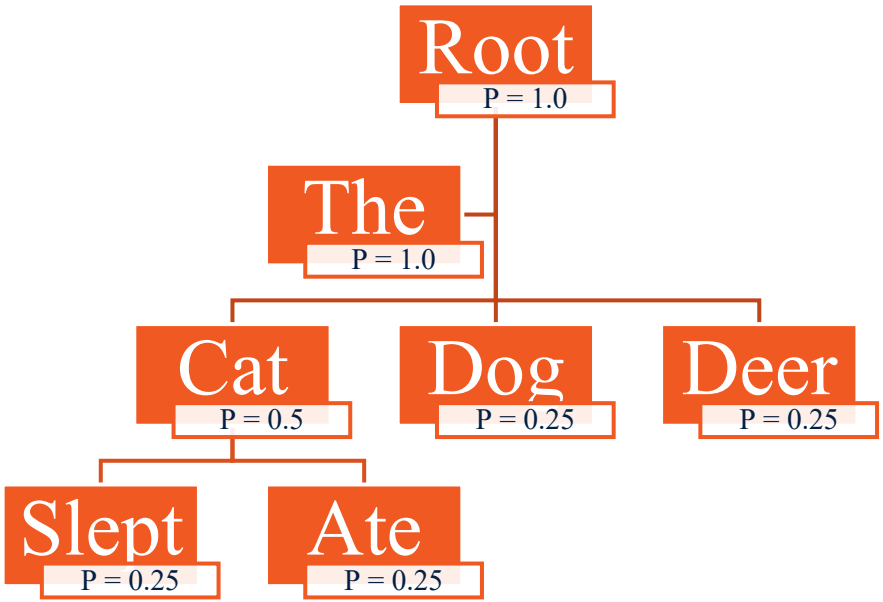
"The goal of lossless compression is to find a compression protocol that encodes a given dataset D and its distribution $\mathrm{p_D}$ with the ***smallest possible expected length*** (L) and allows for a decoding scheme that can perfectly reconstruct the original dataset"

What is the aim when training a LM (Prediction)?

Given some dataset of language (D_{PT}) and its underlying distribution $(\mathrm{p_D})$, can we learn a model whose predicted next token probabilities are representative and well-calibrated to $\mathrm{p_D}$.

The Optimal Compressor <u>is</u> an Optimal Predictor

Compression as a Lens: Separating the Blend



***Remember*. The goal is to explain what property of alignment causes model behavior to return to the base model.**

We've already seen that a language model is just a probability distribution over sequences, their token tree is the same distribution just redrawn as a shape, shortest code length is simply the concept that brings distribution and training loss into the same domain both visually and mathematically. How can compression help us?

Deriving Loss as a Ratio – Compression Rate

A compression protocol, Huffman coding is both provably optimal and lossless.

As we saw, a token tree's probabilities define a code in which each node costs −log p bits, since we have the same objective in compression and prediction, **minimizing the training loss is the same as minimizing expected code length**, so the ideal code length under this protocol is exactly the model's expected loss, which we can write over our tree.

Consider a finite parameter model $\mathrm{p}_\theta(\cdot)$, training on a dataset D, the ideal code length using Huffman coding of a random response x compressed by the model can be expressed as:

$$\mathbb{E}\left[L_{p_\theta}(x)\right] = \left\lceil \frac{|x|}{d} \right\rceil \left(-\sum_{l=1}^{d} \sum_{j=1}^{2^{l-1}} p_{lj} \log p_{lj} \right)$$

Deriving Loss as a Ratio – Compression Rate

$$\mathbb{E}\left[L_{p_\theta}(x)\right] = \left\lceil \frac{|x|}{d} \right\rceil \left(-\sum_{l=1}^{d} \sum_{j=1}^{2^{l-1}} p_{lj} \log p_{lj} \right)$$

Addressing the blending problem:

Since training and alignment involve multiple datasets with different and independent distributions, we have to represent the aligned distribution under a joint compression scenario for multiple datasets. For N pairwise disjoint datasets $\mathrm{D}_1, \ldots, \mathrm{D}_\mathrm{N}$, the node weights $\mathrm{p}_{\mathrm{lj}}^{\mathrm{D}}$ of the token tree of their union is

$$p_{lj}^{D} = \frac{\sum_{i=1}^{N} p_{lj}^{D_i} \, |D_i|}{\sum_{i=1}^{N} |D_i|}$$

UT San Antonio

Deriving Loss as a Ratio – Compression Rate

Thus, the compression rate on specific datasets can be defined accordingly.

$$\gamma_{p_\theta}^{D_i} = \mathbb{E}_{x\sim P_i}\left[\frac{L_{p_\theta}^{D_i}(x)}{|x|}\right] = \Theta\left(\frac{-\sum_{l=1}^{d}\sum_{j=1}^{2^{l-1}} p_{lj}^{D_i}\log p_{lj}^{D}}{d}\right)$$

And so, leveraging our token tree analogy and the Huffman Coding Protocol. Minimizing loss is equivalent to minimizing the compression rate when models act as a lossless compressor.

This isn't just a proxy saying how good the model is.

Because the compression rate considers a joint compression scenario (multiple datasets), the compression rate says how good is the model specifically at predicting this one dataset, given that it was trained jointly with others. We need this. Why?

Compression Rate as a Comparison Tool

Because the aligned model is trained jointly on several datasets, we can measure its compression rate on each one separately. Tracking these rates at different stages of adversarial fine-tuning shows how well the aligned model is holding up. But can we compare them directly, with no further adjustment? **No.**

Consider the various token trees representing the distribution spaces of datasets $\mathrm{D}_1, \ldots, \mathrm{D}_\mathrm{N}$. A tree with more leaves spreads probability over more outcomes, so a model that knows nothing about that dataset pays more bits, about log M for M leaves. A higher compression rate may therefore reflect a larger tree rather than a worse model. To compare performance on datasets fairly, we must first remove this size effect by normalizing the compression rate.

$$\gamma_{p_\theta}^{D_i/D} = \gamma_{p_\theta}^{D_i} - \log M$$

Naming the Phenomenon

We have seen that the normalized compression rate can be used to compare how a model fits a distribution. And we need this to quantify relationally how aligned models revert back to their base models. But first, we need to define the observations empirically observable.

Inverse alignment: Given a language model, aligned on a dataset to produce an aligned model. Applying a separate dataset, such that the aligned model returns to its pre-aligned state.	*Elasticity:* Consider a language model and its aligned transformation. Elasticity of a language model exists, if there exists an algorithmically simple inverse operation and a dataset that can cause inverse alignment.

Elasticity

Quantity	Result	Meaning
$\frac{d\gamma_{p\theta}^{D_p/D}}{dl}$	< 0	The model compresses pre-training data better perturbation grows
$\frac{d\gamma_{p\theta}^{D_a/D}}{dl}$	> 0	The model compresses alignment data worse
$\frac{d\gamma_{p\theta}^{D_a/D}}{dl} = \Theta(-k\frac{d\gamma_{p\theta}^{D_p/D}}{dl})$	$k = \lvert D_p\rvert/\lvert D_a\rvert >> 1$	Alignment's rate moves about k times faster than pre-trainings

Theorem 4.2 (*Elasticity* of Language Models). *Consider the pre-training dataset* $\mathcal{D}_p = \bigcup_{i=1}^{3} \mathcal{D}_i$, *the alignment dataset* $\mathcal{D}_a$, *and the perturbation dataset* $\mathcal{D}_t$, *with the model* $p_{\boldsymbol{\theta}}(\cdot)$ *trained on* $\mathcal{D} = \mathcal{D}_p \cup \mathcal{D}_a \cup \mathcal{D}_t$. *Assume that* $\mathcal{D}_a \overset{d}{\sim} \mathcal{D}_2$, $\mathcal{D}_t \overset{d}{\sim} \mathcal{D}_3$, *and* $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$ *are each distributed according to a Pareto mass distribution (Newman, 2005). As the perturbation data volume* $|\mathcal{D}_t|$ *varies,* $\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_p/\mathcal{D}}$ *and* $\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_a/\mathcal{D}}$ *satisfy:*

$$\frac{d\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_a/\mathcal{D}}}{dl} = \Theta\left(-k\frac{d\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_p/\mathcal{D}}}{dl}\right), \quad \frac{d\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_p/\mathcal{D}}}{dl} < 0, \frac{d\gamma_{p_{\boldsymbol{\theta}}}^{\mathcal{D}_a/\mathcal{D}}}{dl} > 0, \tag{2}$$

where $l = \frac{|\mathcal{D}_t|}{|\mathcal{D}_a|} \ll 1$, $k = \frac{|\mathcal{D}_p|}{|\mathcal{D}_a|} \gg 1$, *and* $\{\mathcal{D}_i\}_{i=1}^{3}$ *are datasets of equal cardinality.*

UT San Antonio

Evidence of Elasticity

The theorem demonstrated that the normalized compression rate change of different datasets is inversely proportional to their sizes under perturbations. A significant size difference between the pre-training and alignment datasets causes rapid performance degradation on the alignment dataset due to the inverse relationship spanning several orders of magnitude.

How can we analogize this relationship?

Springs Model Elasticity

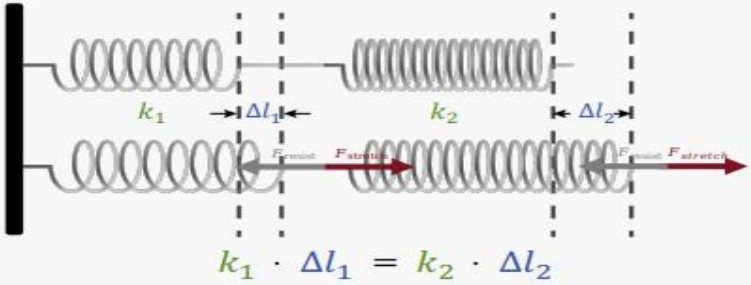
The inverse relationship between normalized compression rate and dataset volume mathematically behaves like a spring.

So, when we think of a spring, we can analogize its properties to how language models behave during alignment and post-alignment fine-tuning.

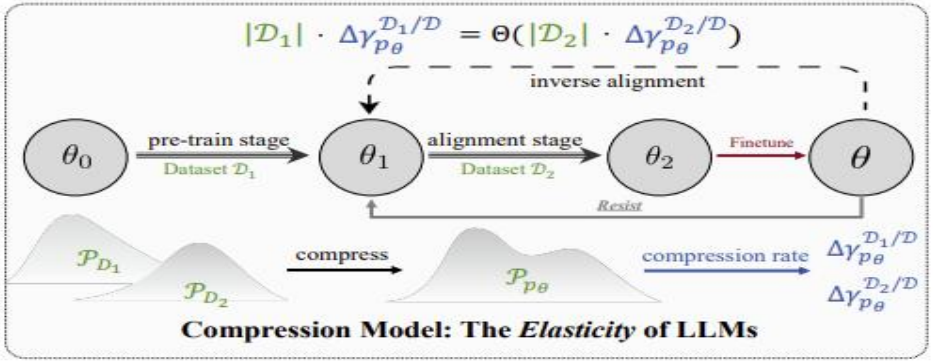
How is aligning a model similar to stretching a spring?

Our Contribution: The *Elasticity* of LLMs

Language models, fine-tuned with perturbations, exhibit an inverse relationship between normalized compression rate changes and dataset volume, akin to that of a series spring system, revealing the *elasticity* of LLMs.



Physical Model: The Hooke's Law



Compression Model: The *Elasticity* of LLMs

[Experiments and Analysis] – Do the theorems hold up?

The question is does *elasticity* cause a resistance to alignment?

To evaluate this the authors attempt to measure two phenomena under elasticity:

- **Resistance** for **Pre-Trained Models**. Models tend to maintain the original distribution and resist alignment;
- **Rebound** for **Post-Trained Models**. Finetuning in the opposite direction of post-training (e.g., safe vs. unsafe) causes post-trained models to return quickly to the pretraining distribution.

They evaluate existence of resistance by testing a phenomena (forward alignment, and inverse alignment). They evaluate rebound through quantitative observation of the effects of model-reverse fine-tuning

[Experiments and Analysis] – Datasets

Alpaca

- An instruction-following dataset: prompts paired with model-generated responses, built to teach a model to follow general instructions (Stanford's self-instruct-style dataset, built on OpenAI's GPT outputs).

TruthfulQA

- A benchmark/dataset of questions designed to test whether models give factually accurate answers rather than repeating common misconceptions or falsehoods.

Beavertails

- A human-preference safety dataset that labels responses as "safe" or "unsafe," built specifically to support safety alignment research.

IMDb

- A movie-review dataset, commonly used in sentiment analysis, where reviews are labeled positive or negative.

[Experiments and Analysis] – Resistance

Experiment Design:

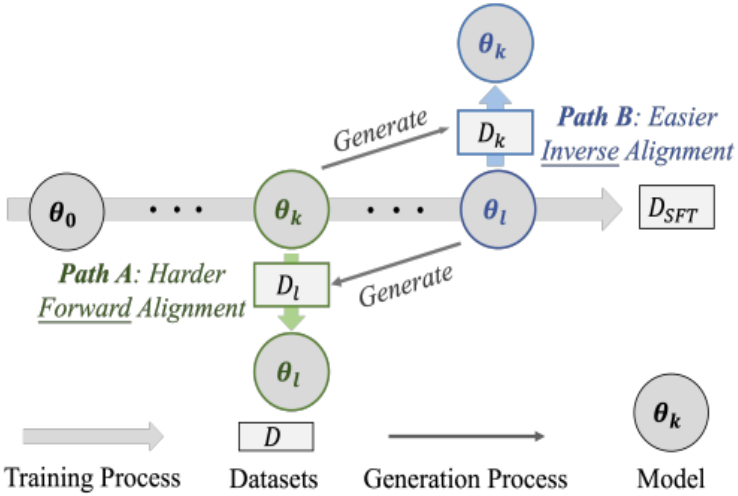
- Run one epoch of SFT on a pre-trained LLM (params θ_0), saving snapshots $\{\theta_1, \theta_2, \ldots, \theta_n\}$
- Take two snapshots θ_k and θ_l ($k < l$), collect each one's responses on held-out prompts → datasets D_k and D_l
- Forward alignment (Path A): train θ_k on D_l
- Inverse alignment (Path B): train θ_l on D_k
- Compare training loss between the two paths

Setup:

- Datasets: Alpaca, TruthfulQA, Beavertails (mapping to helpful/honest/harmless)
- Each split into 3 equal parts → snapshots $\{\theta_1, \theta_2, \theta_3\}$
- Base models: Llama2-7B, Llama2-13B, Llama3-8B

[Experiments and Analysis] – Resistance

Methodology



Result

Datasets	Base Models	$\theta_2 \to \theta_1$ vs. $\theta_1 \to \theta_2$		$\theta_3 \to \theta_2$ vs. $\theta_2 \to \theta_3$		$\theta_3 \to \theta_1$ vs. $\theta_1 \to \theta_3$	
Alpaca	Llama2-7B	0.1589 ↓	0.2018 ↑	0.1953 ↓	0.2143 ↑	0.1666 ↓	0.2346 ↑
	Llama2-13B	0.1772 ↓	0.1958 ↑	0.2149 ↓	0.2408 ↑	0.1835 ↓	0.2345 ↑
	Llama3-8B	0.2540 ↓	0.2573 ↑	0.2268 ↓	0.3229 ↑	0.2341 ↓	0.2589 ↑
Truthful	Llama2-7B	0.1909 ↓	0.2069 ↑	0.1719 ↓	0.1721 ↑	0.2011 ↓	0.2542 ↑
	Llama2-13B	0.1704 ↓	0.1830 ↑	0.1544 ↓	0.1640 ↑	0.1825 ↓	0.2429 ↑
	Llama3-8B	0.2118 ↓	0.2256 ↑	0.2100 ↓	0.2173 ↑	0.2393 ↓	0.2898 ↑
Safe	Llama2-7B	0.2730 ↓	0.2809 ↑	0.2654 ↓	0.2691 ↑	0.2845 ↓	0.2883 ↑
	Llama2-13B	0.2419 ↓	0.2439 ↑	0.2320 ↓	0.2327 ↑	0.2464 ↓	0.2606 ↑
	Llama3-8B	0.2097 ↓	0.2156 ↑	0.2008 ↓	0.2427 ↑	0.2277 ↓	0.2709 ↑

[Experiments and Analysis] – Rebound

Experiment Design:

- Train snapshots $\{\theta_1, \theta_2, \ldots, \theta_n\}$ from base model θ_0 using positive/safe data at varying volumes
- Then apply inverse fine-tuning using negative/unsafe data
- Track performance before and after the inverse fine-tuning

Tasks and Datasets used:

- Positive-generation task: IMDb (positive vs. negative movie review generation)
- Safe-dialogue task: Beavertails (safe vs. unsafe single-turn conversation)
- Positive/safe sample sizes tested: {1000, 2000, 5000, 10000}
- Negative/unsafe sample sizes tested: {100, 200, 500, 1000, 2000}

[Experiments and Analysis] – Rebound

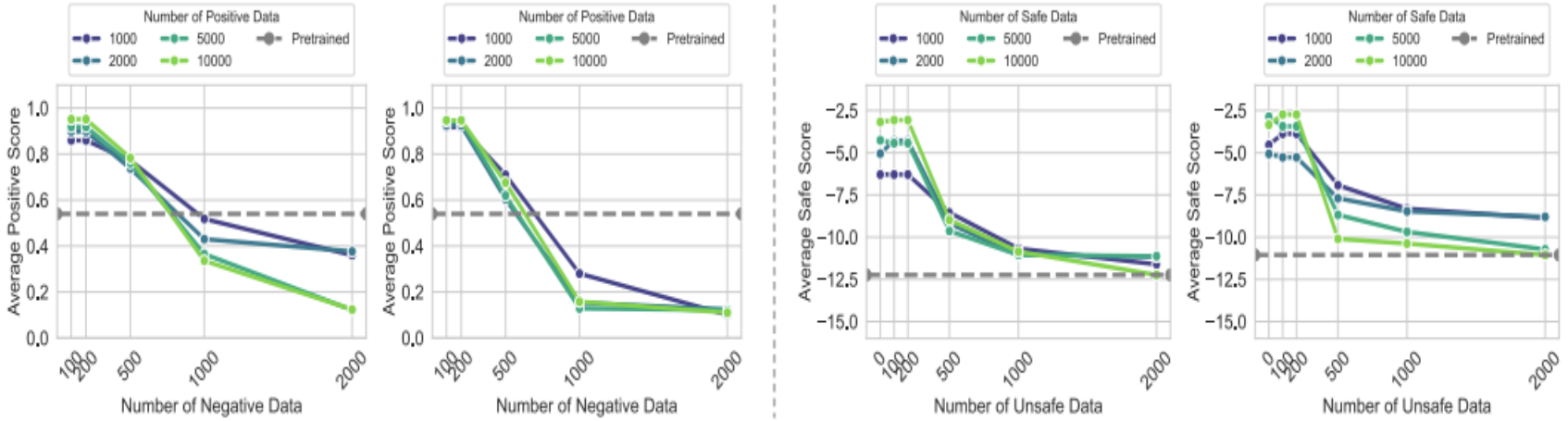


Figure 4: **Experimental results for validating the existence of *rebound* (left: IMDb, right: Beavertails).** The left part of each sub-figure is the performance of Gemma-2B while the right is Llama2-7B, respectively. Models trained with more **positive** data initially perform better but perform worse after fine-tuning with **negative** data.

[Experiments and Analysis] – Rebound

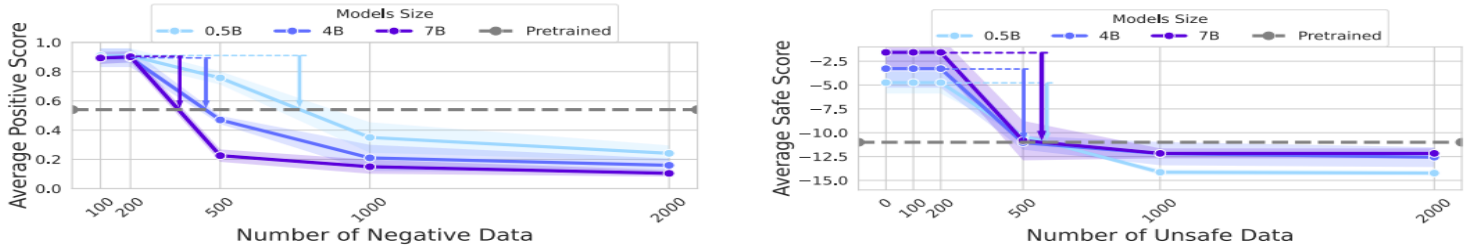


Figure 5: **Experimental results for validating *rebound* increases with model size (left: IMDb, right: Beavertails).** All single line covers positive data volume settings as Figure 4, with shadow denoting std. As the model size increases, the performance of the aligned model deteriorates more rapidly after fine-tuning with **negative** data.

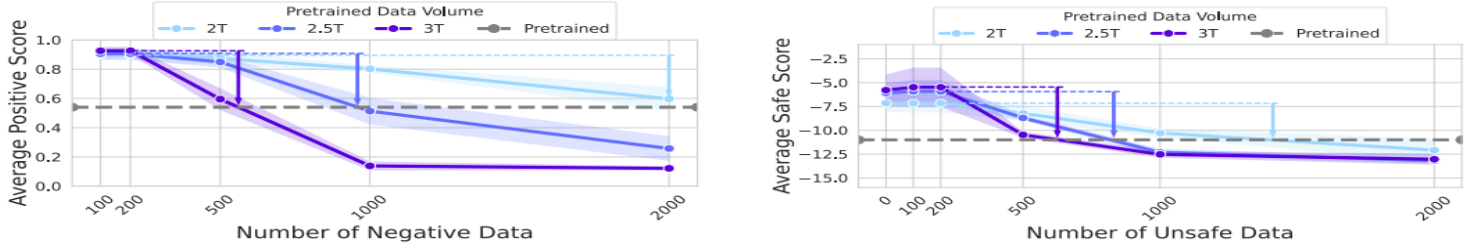


Figure 6: **Experimental results for validating *rebound* increases with pre-training data volume. (left: IMDb, right: Beavertails).** Each line covers positive data settings as Figure 4, with shadow denoting std. As pre-training data volume increases, aligned model performance deteriorates more rapidly after fine-tuning with **negative** data.